

JosephusProblem-master

340/JosephusProblem-master/README.md

CSCI 340 - Data Structures and Algorithm Analysis

Data Structure: Linked List

Programming Focus: STL List, Recursion, Generator, Iterator, and Iterator invalidation

The Josephus Problem

The problem is known as the **Josephus Problem** (or Josephus permutation) and postulates a group of people of size $N \geq 1$ are standing in a circle waiting to be eliminated. Counting begins at a specified point in the circle and proceeds around the circle in a specified direction. After a specified number of $M \geq 1$ people are counted, the M^{th} person in the circle is eliminated. The procedure is repeated with the remaining people, starting with the next person, going in the same direction and counting the same number of people, until only one person remains.

For example, suppose that $M = 3$ and there are $N = 5$ people named **A**, **B**, **C**, **D** and **E**. We count three people starting at **A**, so that **C** is eliminated first. We then begin at **D** and count **D**, **E** and back to **A**, so that **A** is eliminated next. Then we count **B**, **D** and **E**, and finally **B**, **D** and **B**, so that **D** is the one who remains last. (See also https://en.wikipedia.org/wiki/Josephus_problem)

For this computer assignment, you are to write and implement a C++ program to simulate and solve the Josephus problem.

The input to the program is:

- The initial number of people N (default = 41)
- The counting distance M (default = 3)
- The frequency to print a dump of the list of remaining names (default = 13)
- The number of columns to display when printing the list of remaining people (default = 12)

When the elimination logic is running, print the list of remaining names each time

`print_frequency` eliminations have been performed. (See the various **josephus.refout-xxx** examples)

After all eliminations have completed, print the list one last time to show the single surviving name.

Programming Notes:

- `void print_list(const list &collection, const unsigned &eliminations, int num_cols)`
 Print a heading including the number of `eliminations` and the contents of `collection` with `num_cols` elements on each line. When `eliminations` is zero, print Initial group of people as the table heading. Otherwise, print After eliminating N people where N is the number of `eliminations`.

- `void print_underlined_string(const string &message)`
Print the given `message` with an underline that matches the length of the message text. Use this to print the heading for the list in `print_list()`.
- The `main()` must create an empty `std::list` container and then call `std::generate_n()` with a `std::back_inserter` on your, currently empty, list to insert the IDs/names for the initial N number of people by using the provided generator class. Note that the `SEQ` class is provided for use as the last argument to `generate_n()` that you will have to construct passing it the number of people N for for which it will generate names (for example `SEQ(num_people)`). Once the list generated, print it using `print_list()` and initialize an iterator to the first element in your list. Advance your iterator by one each time through then loop until you reach the next element to eliminate. After each elimination, if it is time, call `print_list()` to see an incremental state of the elimination process. Your elimination loop continues until only one person remains in the list where you will print the list one last time to see the last survivor.

Assignment Notes:

- Your elimination logic must be implemented by using an iterator and **only** advancing it by using the pre-increment `++it` operator. You must **not** re-start and advance from the beginning of the list container to locate each element to eliminate (doing so would be inefficient on an linked list.) In other words, you must advance the iterator to the a position to eliminate, call `erase(it)` to remove it and then continue with the next element position. Be careful not to use an iterator that has been invalidated by `erase()` ! (See how `std::erase()` works and what it returns to remedy this invalidation problem here: <https://en.cppreference.com/w/cpp/container/list/erase>)
- See https://en.cppreference.com/w/cpp/algorithm/generate_n for details on how to use `std::generate_n()`.
- Note that the given reference output file names include the command-line parameters used to create them. For example, the file named `josephus.refout-n25-m2` was created by running this command:

```
./josephus -n25 -m2 > ./josephus-n25-m2
```

- You are not allowed to use any I/O functions from the C library, such as `scanf` or `printf`. Instead, use the I/O functions from the C++ library, such as `cin` or `cout`.
- Add documentation to the appropriate source files as discussed in your class.
- Prepare your Makefile (you need to construct and add Makefile) so that the TA only needs to invoke the command `make` to compile your source file and produce the executable file **josephus**. Make sure you use exactly the same file names specified here, i.e. **josephus.cc** and **josephus**, in your Makefile, otherwise your **submission will get 0 points**.
- For full credit your output must exactly match the reference output as verified with the `diff` command.

When your program is ready for grading, ***commit*** and ***push*** your local repository to remote git classroom repository and follow the ***Assignment Submission Instructions***.